

Introducción

En este informe se presenta la documentación técnica del sistema distribuido pedido para el análisis de lavado de activos en base a una gran cantidad de transacciones bancarias.

Se presenta con el formato de Vistas de arquitectura 4+1, separando en:

- Escenarios
- Vista Lógica
- Vista de desarrollo
- Vista de procesos
- Vista física

Escenarios y casos de uso

El cliente envía un lote de transacciones tal que se apliquen 5 queries sobre los datos:

1. Detección de transferencias de bajo monto

- **Escenario:** Identificar operaciones directas que involucren montos menores a 50 USD.
- **Resultado esperado:** El registro de las transacciones que cumplan esta condición, incluyendo la información del cliente, los bancos involucrados, las cuentas y el monto.

2. Extracción de transacciones máximas por banco

- **Escenario:** Registrar la transferencia de mayor valor histórico procesada por cada banco.
- **Resultado esperado:** El registro consolidado de la transacción con el monto máximo global absoluto para cada uno de los bancos procesados.

3. Filtrado dinámico basado en promedios históricos

- **Escenario:** Detectar transacciones recientes (del período 2022-09-06 al 2022-09-15) cuyo valor sea menor al 1% del promedio histórico (calculado sobre el período 2022-09-01 al 2022-09-05) para su respectivo formato de pago.
- **Resultado esperado:** Únicamente las transacciones del período reciente que cumplan con esta regla matemática de desviación respecto a su promedio.

4. Detección de patrones de triangulación

- **Escenario:** Identificar esquemas de transferencia repetitivos (más de 5 ocurrencias) que sigan el patrón de triangulación (De profundidad 1) dentro del período del 2022-09-01 al 2022-09-05.
- **Resultado esperado:** Una alerta o reporte detallando la cuenta y el banco de origen para aquellos pares que superen el umbral de 5 repeticiones de dicho ciclo.

5. Conteo de transferencias de bajo valor (ACH/Wire)

- **Escenario:** Contabilizar el volumen total de transacciones menores a 1 USD ejecutadas bajo los formatos "Wire" o "ACH" en el período del 2022-09-01 al 2022-09-05.
- **Resultado esperado:** Un contador final unificado por cliente que represente la suma total de las operaciones que cumplen con estos criterios.

El sistema debe soportar la consulta paralela de múltiples clientes.

Vista Lógica

Para implementar las 5 queries, se propone una implementación basada en map reduce. La implementación de cada query es independiente.

Query 1: Detección de transferencias de bajo monto

- **Processor:** Evalúa cada transacción de manera individual y sin estado, aplicando el filtro estricto de que el monto sea menor a 50 USD. Si la condición se cumple, se emite el registro directamente hacia la etapa final, haciendo un bypass de la etapa de agrupación.
- **Aggregator:** No aplica. Al ser una operación lineal sobre registros independientes, no se requiere agrupación ni cálculo de estado.
- **Join:** No ejecuta lógica de cruce; actúa únicamente para la tarea de control, enviando el EOF final al cliente cuando todos los nodos de procesamiento terminaron.

Query 2: Extracción de transacciones máximas por banco

- **Processor:** Mantiene un estado temporal en memoria mediante un diccionario. Evalúa cada entrada para actualizar el monto máximo procesado localmente por cada banco (id_banco). Al recibir la señal EOF, emite únicamente este valor máximo parcial.
- **Aggregator:** Agrupa los máximos parciales recibidos desde los distintos workers utilizando el hash del banco. Espera recibir la señal EOF de la totalidad de los nodos de procesamiento para comparar estos subtotales y emitir el máximo global absoluto.
- **Join:** Actúa únicamente como colector final, recibiendo el máximo global del Reduce para consolidarlo en la salida hacia el cliente.

Query 3: Filtrado dinámico basado en promedios históricos

- **Processor:** Bifurca lógicamente el flujo según las fechas. Las transacciones del período 01 al 05 se acumulan en memoria para calcular sumas y cantidades por formato de pago, emitiendo un promedio parcial al llegar EOF. Las transacciones del período 06 al 15 se retransmiten inmediatamente en flujo continuo (stream).
- **Aggregator:** Consolida los promedios parciales recibidos de la primera bifurcación. Al llegar todos los mensajes EOF, calcula el promedio global absoluto por cada formato de pago y lo emite.
- **Join:** Funciona como un Join con estado (Stateful Join). Almacena el stream de transacciones recientes en memoria, espera la llegada del promedio global calculado por el Reduce, y finalmente cruza ambos datos para emitir sólo aquellas transacciones cuyo monto sea menor a promedio/100.

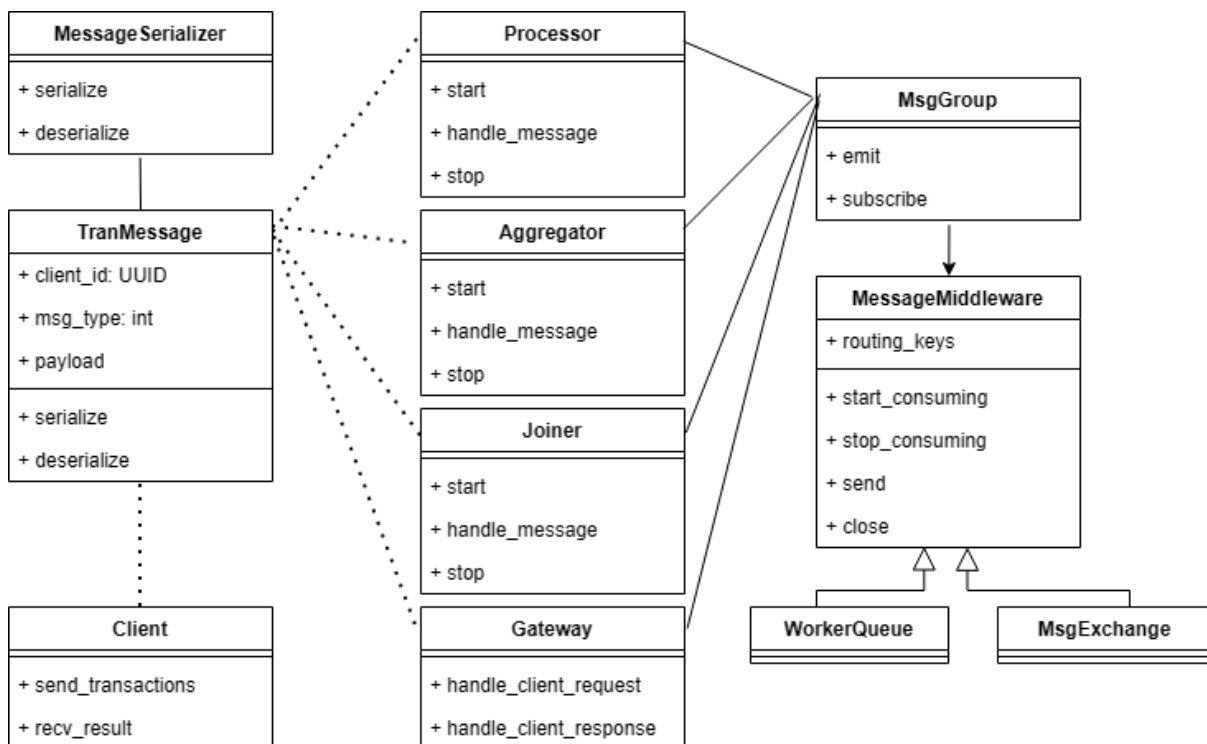
Query 4: Detección de patrones de triangulación

- **Processor:** Ejecuta la duplicación intencional de registros para permitir el análisis de grafos. Por cada transferencia procesada (A,X1), emite dos copias exactas a la red: una particionada mediante el hash de la cuenta de origen (A) y otra mediante el hash de la cuenta destino (X1).
- **Aggregator:** Recibe y agrupa estas transacciones duplicadas. Al coincidir las mitades, ensambla lógicamente la cadena de triangulación formando la estructura A. Luego, emite esta cadena completa particionándola por la clave combinada de origen y destino (hash(A, B)).
- **Join:** Agrupa las cadenas ensambladas por el Reduce. Mantiene un recuento histórico de las repeticiones del par A-B, descarta las cadenas duplicadas generadas por el particionamiento previo, y si el contador supera el umbral estricto de 5 ocurrencias, emite la alerta final a la cola de resultados.

Query 5: Conteo de transferencias de bajo valor (ACH/Wire)

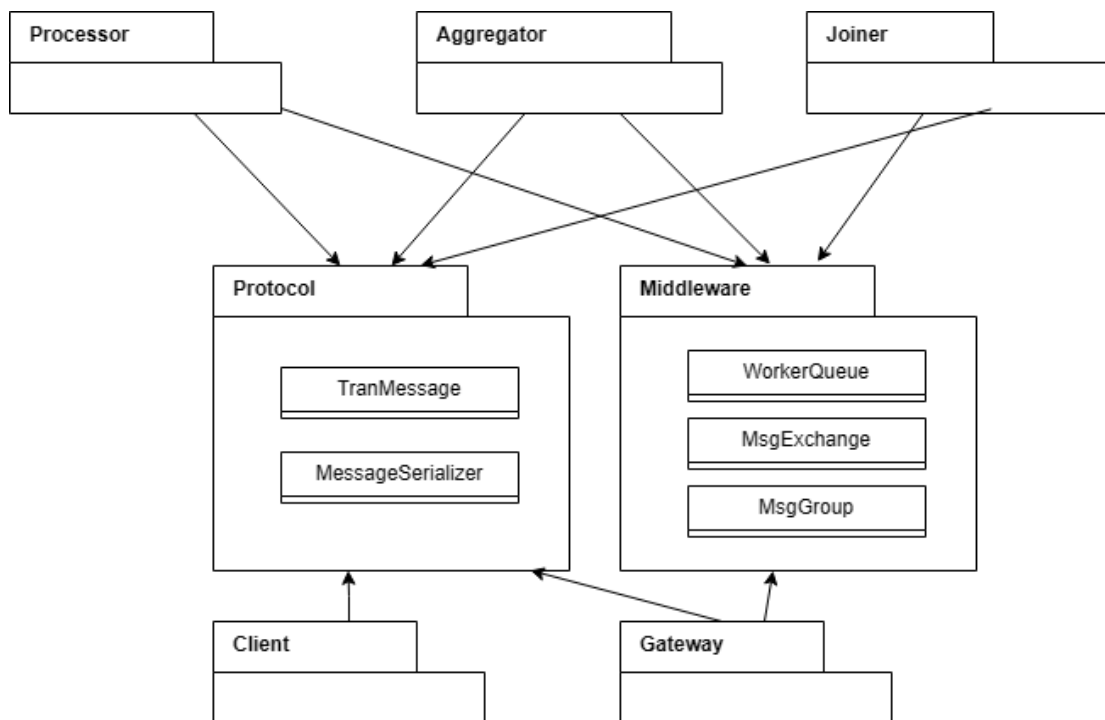
- **Processor:** Aplica un filtro de fecha, formato (Wire/ACH) y monto (< 1 USD). Por cada coincidencia, en lugar de emitir el registro, incrementa un contador numérico local en memoria. Al procesar el archivo completo (EOF), emite el subtotal acumulado, particionado por client_id.
- **Aggregator:** Agrupa los recuentos parciales que llegan de todos los workers para un mismo cliente. Realiza una sumatoria acumulativa simple y, al recibir el EOF de todos los nodos Map, emite la suma total definitiva.
- **Join:** Actúa como colector final sin aplicar cruces, retransmitiendo el contador unificado devuelto por el Reduce hacia la cola de resultados final.

Diagramas de clases



Vista de desarrollo

Diagrama de paquetes



Vista de procesos

Aunque las queries se resuelven en el sistema de forma independiente y en paralelo, decidimos que los nodos de procesamiento y los de agregación son homogéneos, por lo que cualquier nodo puede resolver cualquier query. Y la forma en que distribuimos las tareas hacen que los nodos resuelvan las 5 queries en paralelo para cada transacción entrante.

Ventajas (Pros)

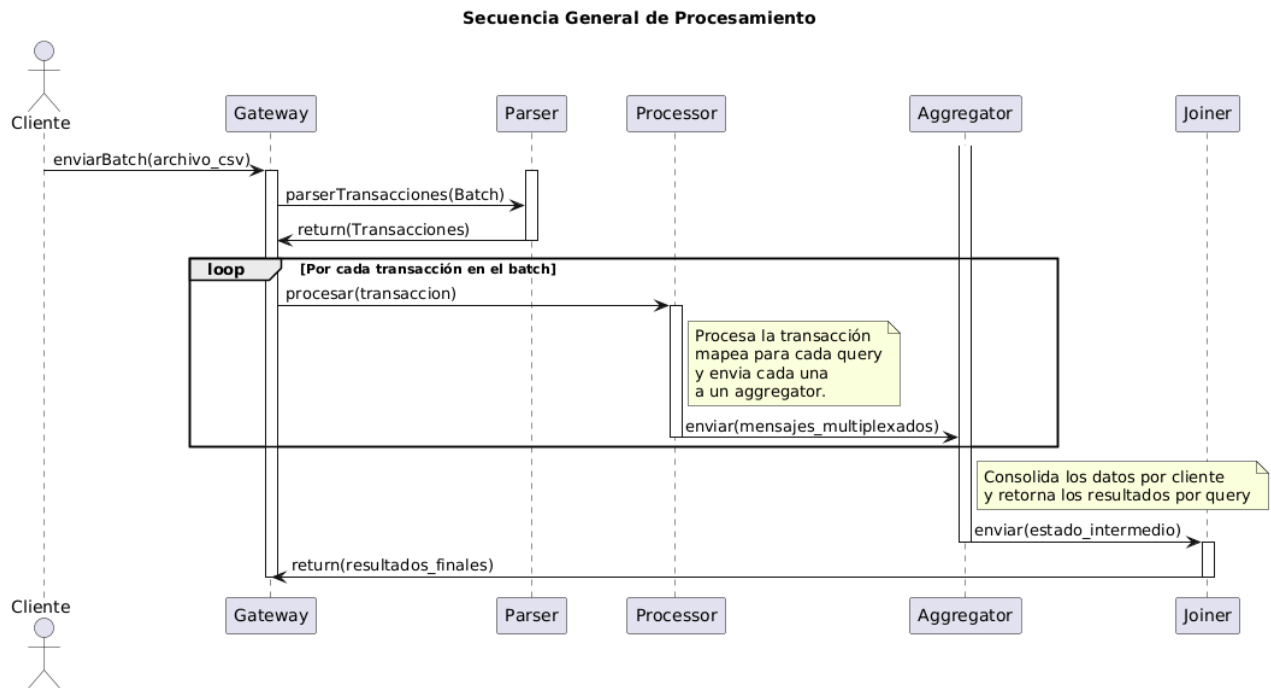
- Eficiencia de Red (Menor I/O)
- Simplicidad de Despliegue
- Balanceo de Carga Equitativo

Desventajas (Contras)

- Falta de Aislamiento, propenso a fallos
- Escalabilidad Rígida: Si el procesamiento de una query demora más que otras, no se puede escalar en forma diferencial
- Acoplamiento de Código, un cambio en una query requeriría desplegar todos los nodos.

Para mitigar estas desventajas, se podría implementar un mecanismo de afinidad en las instancias, que permitiría destinar instancias para uso exclusivo de un tipo de query. Esta implementación debería realizarse en el Middleware, ya que es el responsable de la lógica de grupos

La forma general que toma el sistema desde la vista del cliente es:



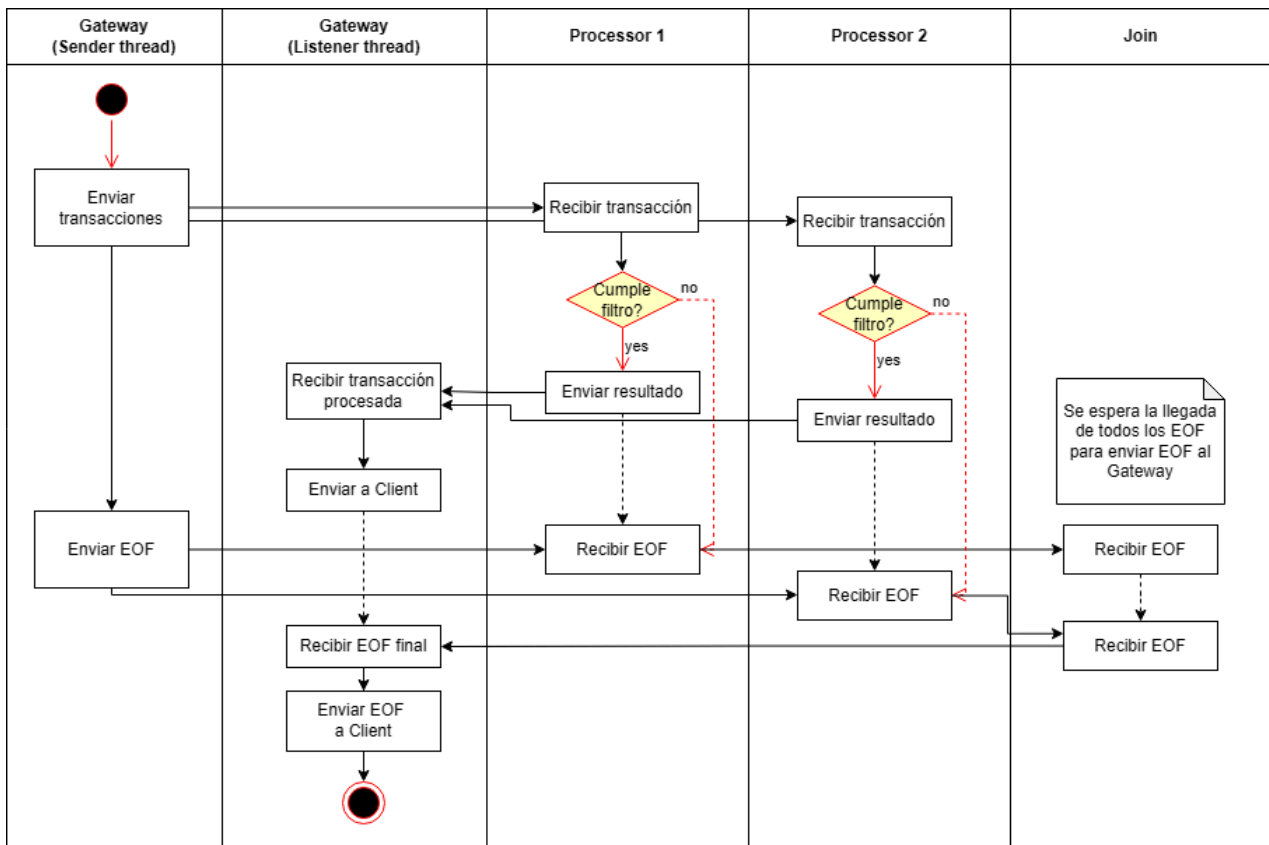
Diagramas de actividades

A pesar de que en todos los casos las transacciones se distribuyen hacia varios nodos de procesamiento, solo se deja explícito en el diagrama para la query 1, ya que complejiza el diagrama y lo hace difícil de leer.

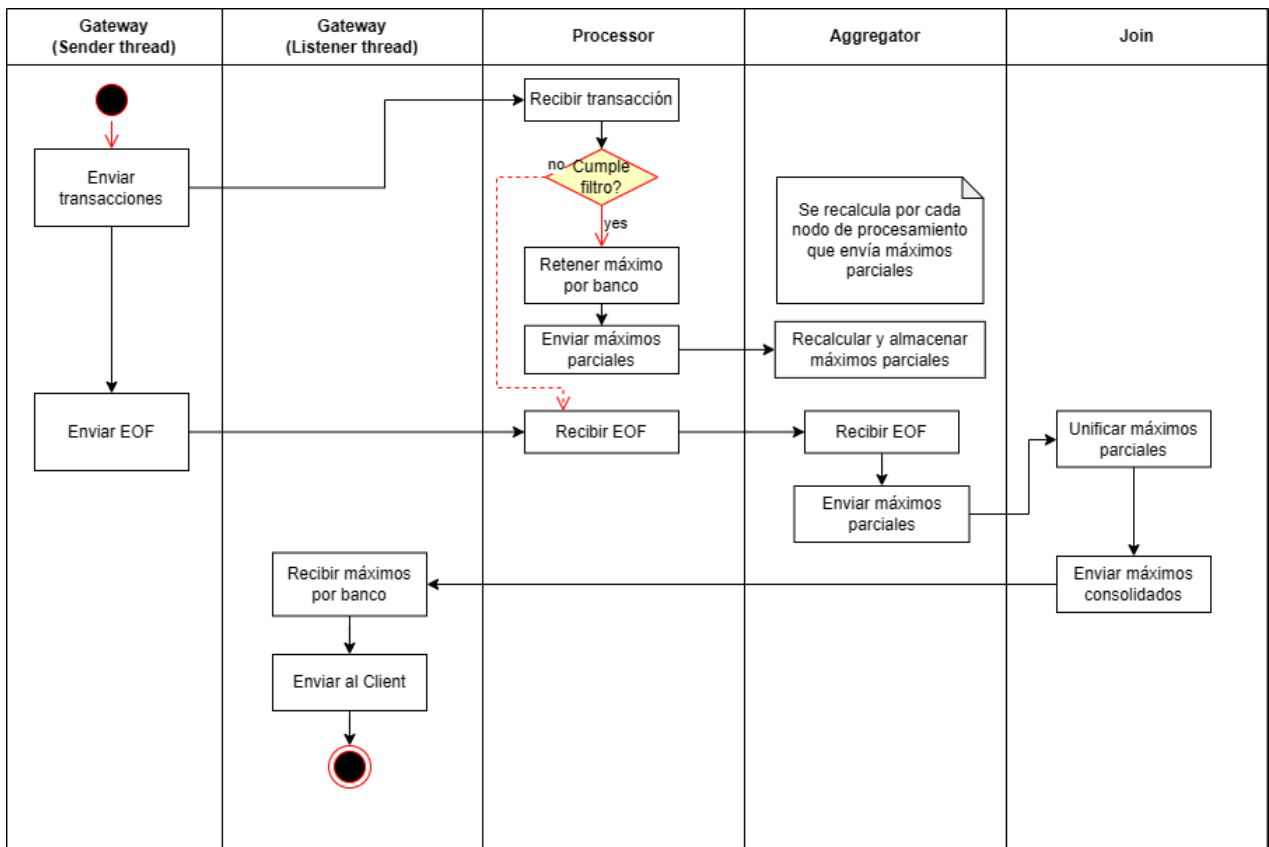
Por la misma razón, no se incluye la comunicación con la instancia que contiene el Middleware, sin embargo, los envíos de datos entre nodos se realizan por medio de colas y exchanges.

- Query 1

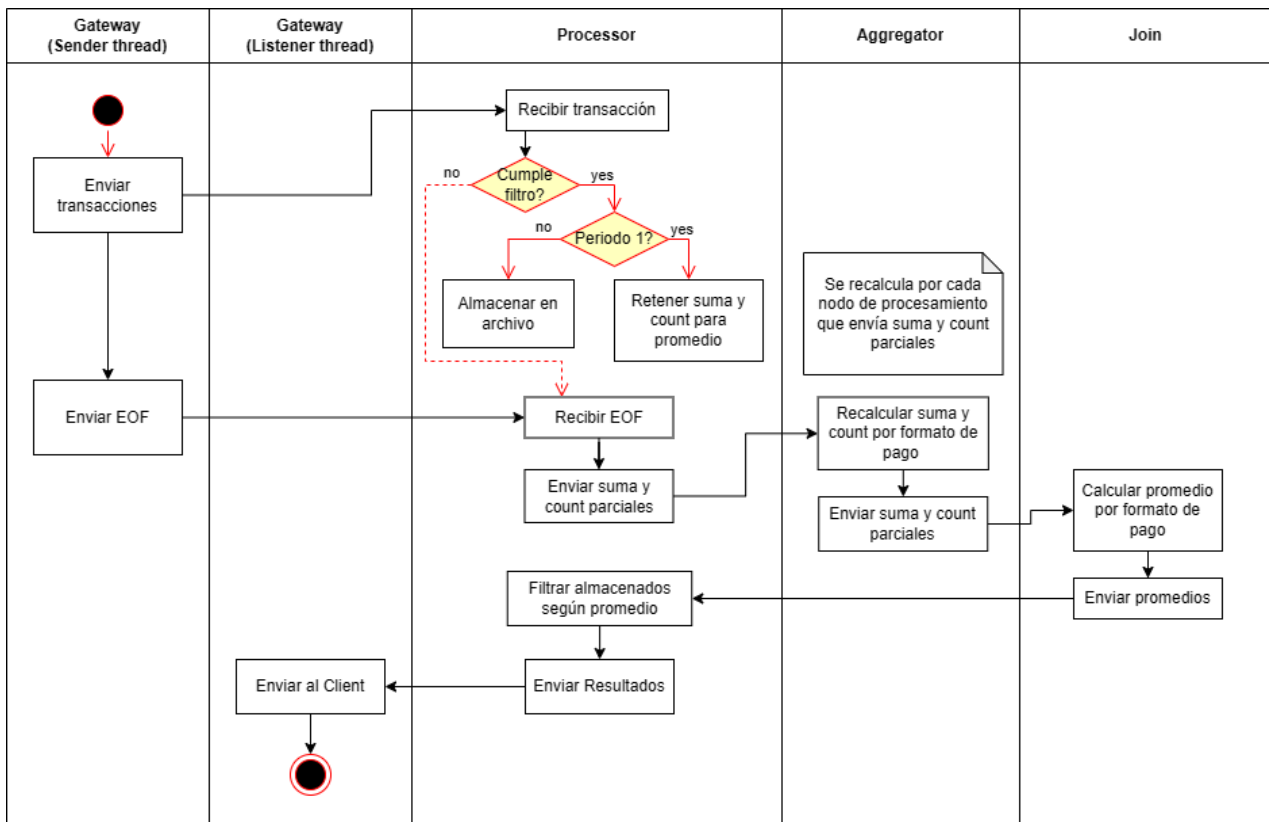
En este caso al no necesitar leer todas las transacciones para empezar a enviar resultados, se envían a medida que se procesan, directo desde el nodo de procesamiento. El joiner solo interviene en la señal EOF.



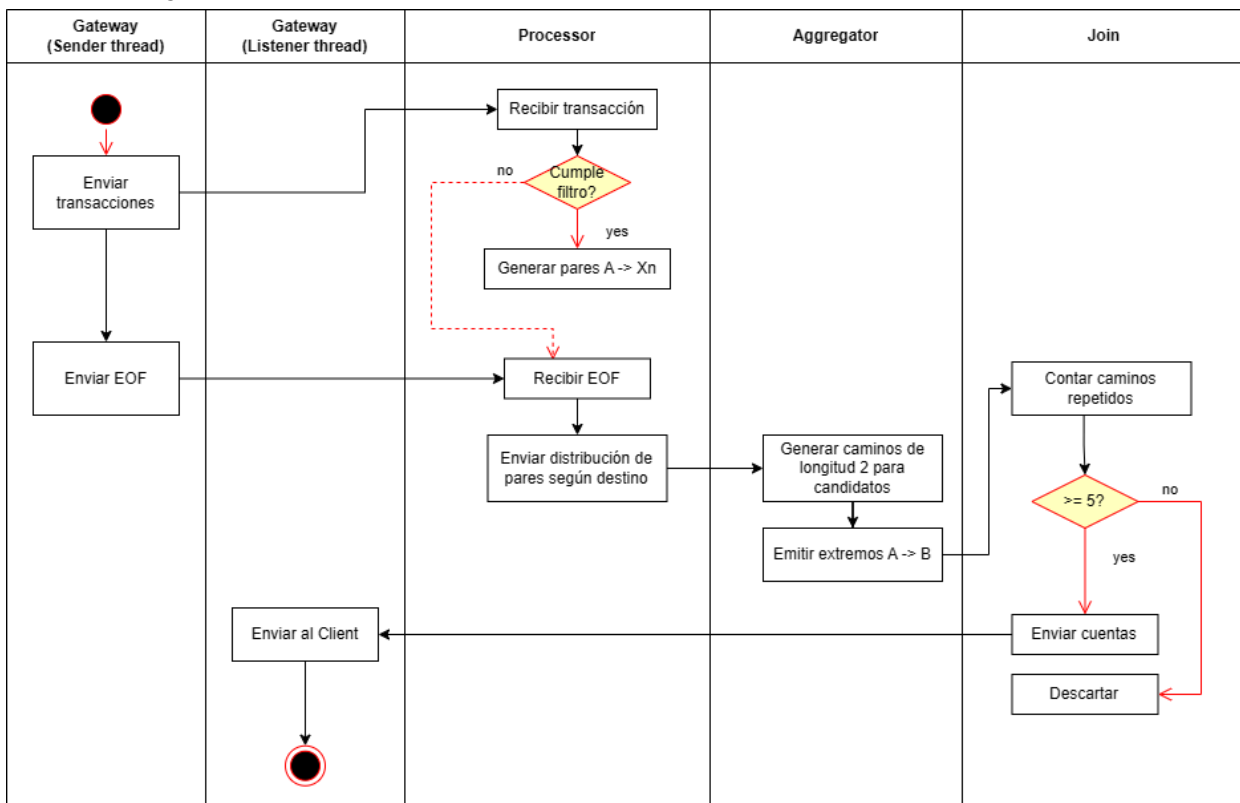
- Query 2



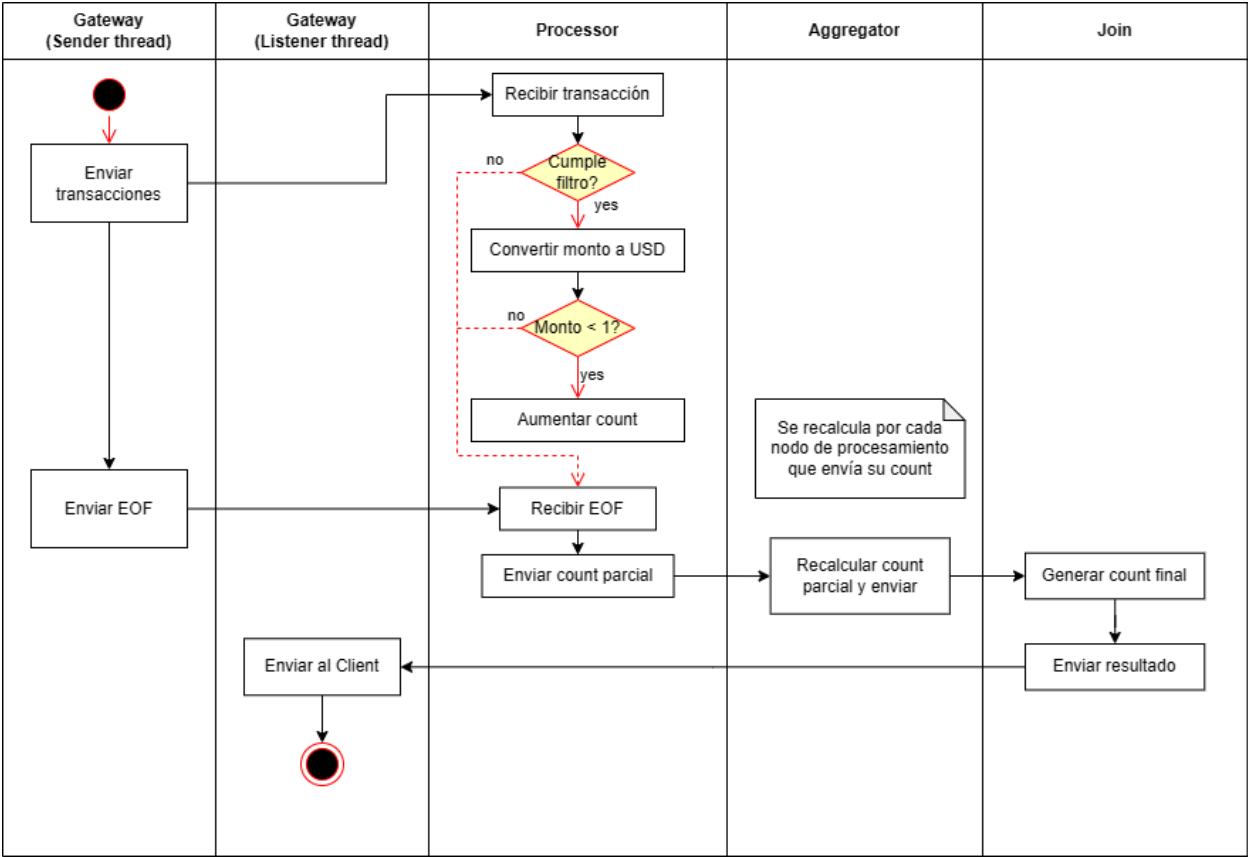
- Query 3



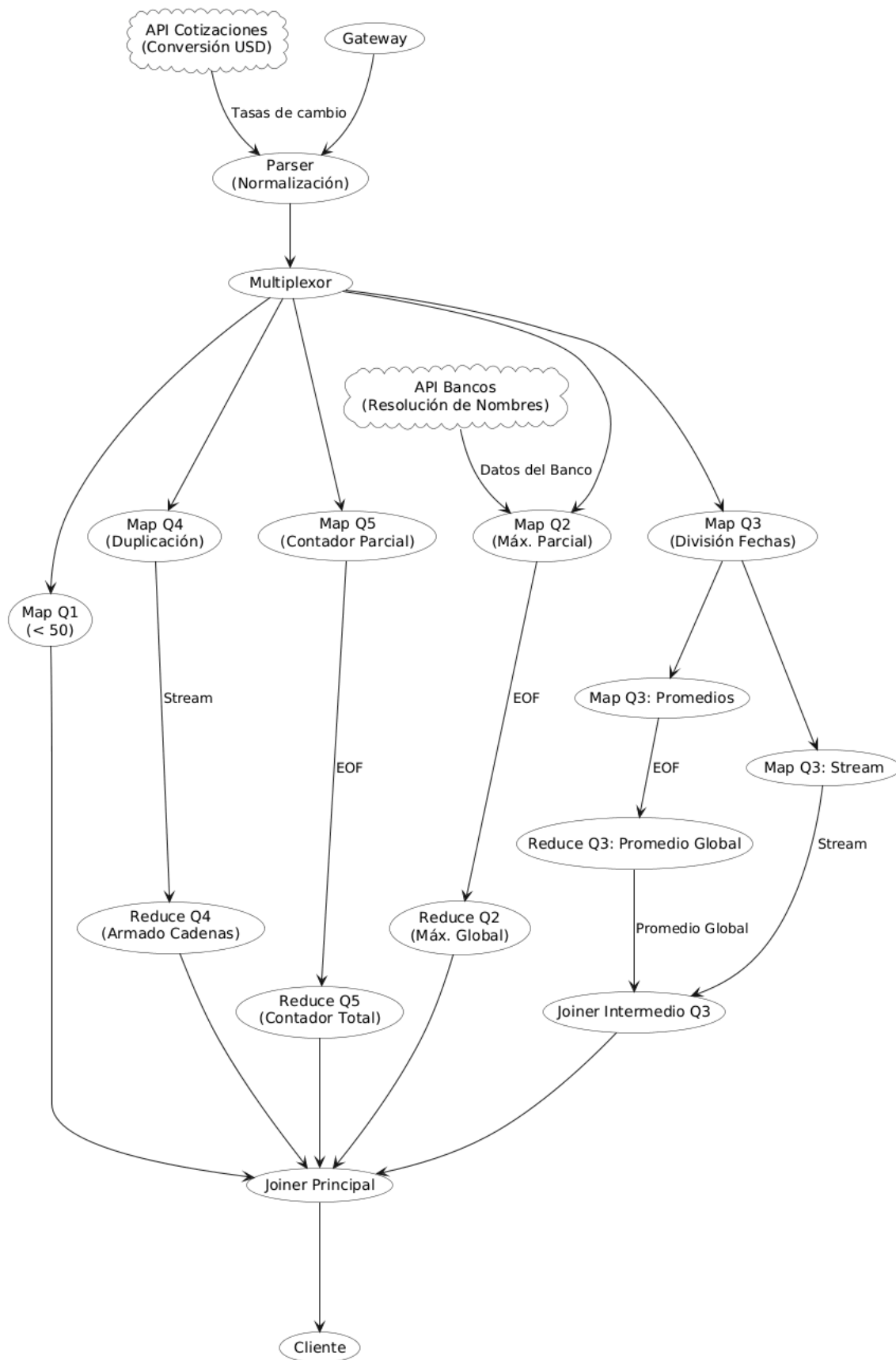
- Query 4



- Query 5



DAG



Vista física

Diagrama de robustez

En este sistema, la gran mayoría del trabajo está concentrado en los nodos de procesamiento y los de agregación. Algunas queries necesitan consolidar datos hacia un resultado final. Esta actividad integradora se delega al join, pero intentando minimizar su carga todo lo posible, ya que por su naturaleza no puede escalar arbitrariamente.

Sin embargo, si fuese necesario, se puede escalar por flujo. Es decir, para distintas queries, o distintos clientes, podría implementarse una derivación a distintas instancias de join.

En el diagrama también se puede observar que los nodos de procesamiento se comunican con una API para la conversión de monedas (Query 5), y tienen un storage temporal de transacciones para utilizar en una etapa posterior (Query 3).

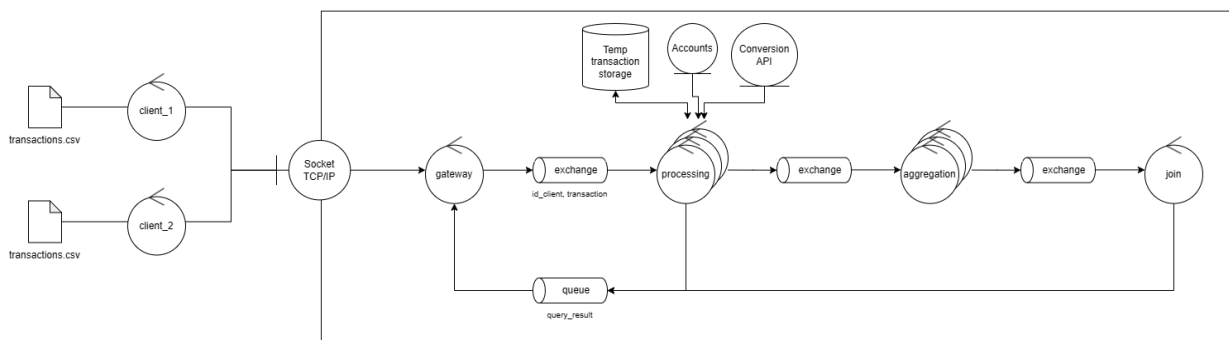


Diagrama de despliegue

